МИКРОСЕРВИСНАЯ АРХИТЕКУТРА

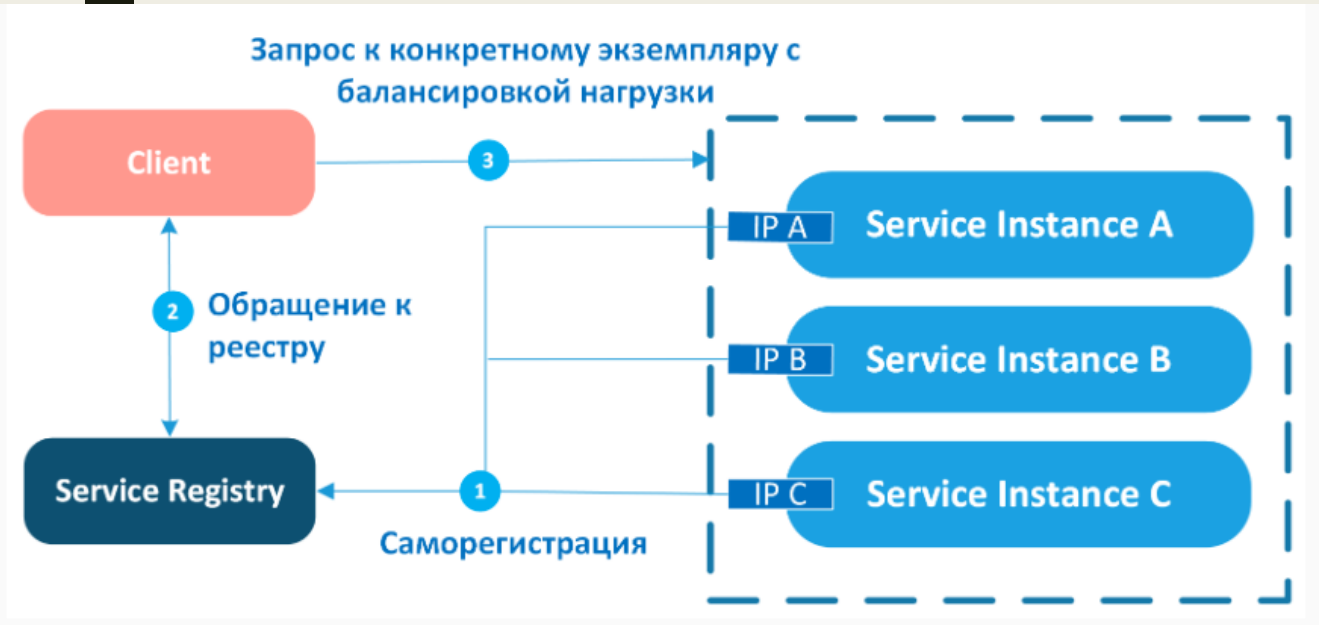
Синхронный обмен сообщениями

Паттерны обнаружения сервисов в микросервисной архитектуре

- Чтобы обменяться сообщениями, один сервис должен знать о существовании другого и наоборот. То есть адрес-порт, куда отправлять сообщения и откуда ждать ответа.
- Так как количество и адреса микросервисов могут меняться динамически, то необходим какой-то механизм автообнаружения сервисов.
- Ключевым компонентом обнаружения микросервисов выступает реестр сервисов (Service Registry) — база данных с информацией о расположении сервисных экземпляров. Когда экземпляры запускаются и останавливаются, информация в реестре обновляется. Но взаимодействовать с реестром сервисов можно двумя путями, которые и легли в основу описанных ниже шаблонов.

https://mcs.mail.ru/blog/26-osnovnyh-patternov-mikroservisnoj-razrabotki

Шаблон «Обнаружение сервисов на стороне клиента» (Client-Side Service Discovery)



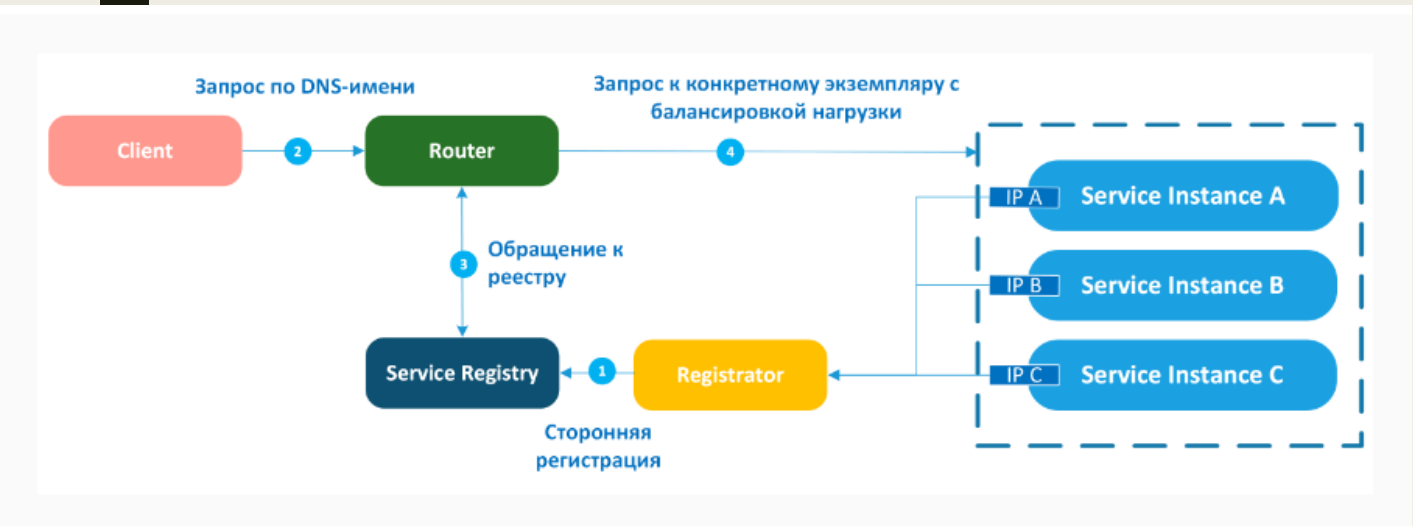
- В этом случае сервисы и их клиенты напрямую взаимодействуют с реестром. Последовательность шагов следующая:

1. Экземпляр сервиса обращается к API реестра, чтобы зарегистрировать свое сетевое местоположение. Он также может предоставить URL-адрес для проверки своей работоспособности (Health Check), который будет использоваться для продления срока его регистрации в реестре.
2. Клиент самостоятельно обращается к реестру сервисов, чтобы получить список экземпляров сервисов. Для улучшения производительности клиент может кэшировать экземпляры сервиса.
3. Клиент использует алгоритм балансировки нагрузки, циклический или случайный, чтобы выбрать конкретный экземпляр сервиса и отправить ему запрос.

Шаблон «Обнаружение сервисов на стороне клиента» (Client-Side Service Discovery)

- Ключевое преимущество обнаружения сервисов на стороне клиента — его независимость от используемой платформы развертывания. Например, если часть ваших сервисов развернута на K8s, а остальные работают в устаревшей среде, то обнаружение на уровне приложения будет лучшим вариантом, так как серверное решение на базе Kubernetes не будет совместимо со всеми сервисами.
- К недостаткам подхода можно отнести необходимость использования различных клиентских библиотек для каждого языка программирования, а иногда и фреймворка. Кроме этого, на вашу команду ложится дополнительная нагрузка по настройке и обслуживанию реестра сервисов.

Шаблон «Обнаружение сервисов на стороне сервера» (Server-Side Service Discovery)



- В этом случае за регистрацию, обнаружение сервисов и маршрутизацию запросов отвечает инфраструктура развертывания. Последовательность шагов следующая:

1. Регистратор, который обычно является частью платформы развертывания, прописывает все экземпляры сервисов в реестре сервисов. По каждому экземпляру сохраняется DNS-имя и виртуальный IP-адрес (VIP).
2. Вместо того чтобы обращаться к реестру напрямую, клиент делает запрос по DNS-имени сервиса. Запрос поступает в маршрутизатор, являющийся частью платформы развертывания.
3. Маршрутизатор обращается к реестру сервисов для получения сетевого расположения экземпляров нужного сервиса.
4. Маршрутизатор применяет балансировку нагрузки, чтобы выбрать конкретный экземпляр сервиса и отправить ему запрос.

Шаблон «Обнаружение сервисов на стороне сервера» (Server-Side Service Discovery)

- Все современные платформы развертывания, включая Docker, Kubernetes и другие, как правило, имеют встроенный реестр и механизмы обнаружения сервисов.
- Основное преимущество паттерна состоит в том, что всеми аспектами обнаружения сервисов занимается сама платформа. Дополнительный код на стороне клиента или сервисов не требуется. Благодаря этому достигается независимость от используемых в приложении языков программирования и фреймворков.
- Недостатком паттерна является невозможность его применения к сервисам, которые развернуты вне основной платформы, реализующей механизмы обнаружения. Несмотря на это ограничение, рекомендуется использовать обнаружение сервисов на стороне сервера всюду, где это осуществимо.

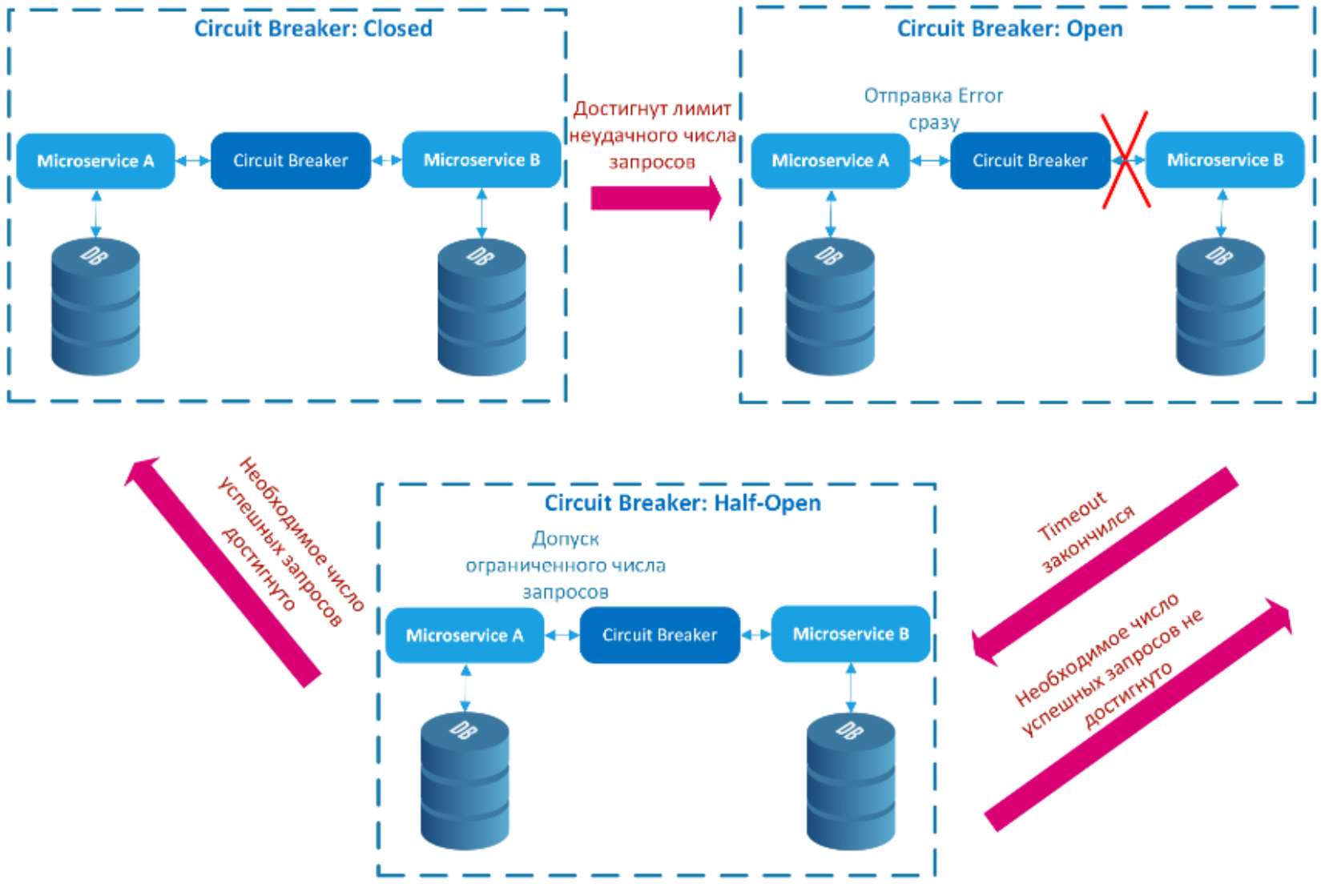
Паттерны повышения отказоустойчивости

- Обмен сообщениями должен быть надежным и выход из строя одного сервиса не должен приводить к отказу всей системы.
- Посмотрим, что можно сделать для повышения надежности микросервисной архитектуры

Шаблон «Автоматический выключатель» (Circuit Breaker)

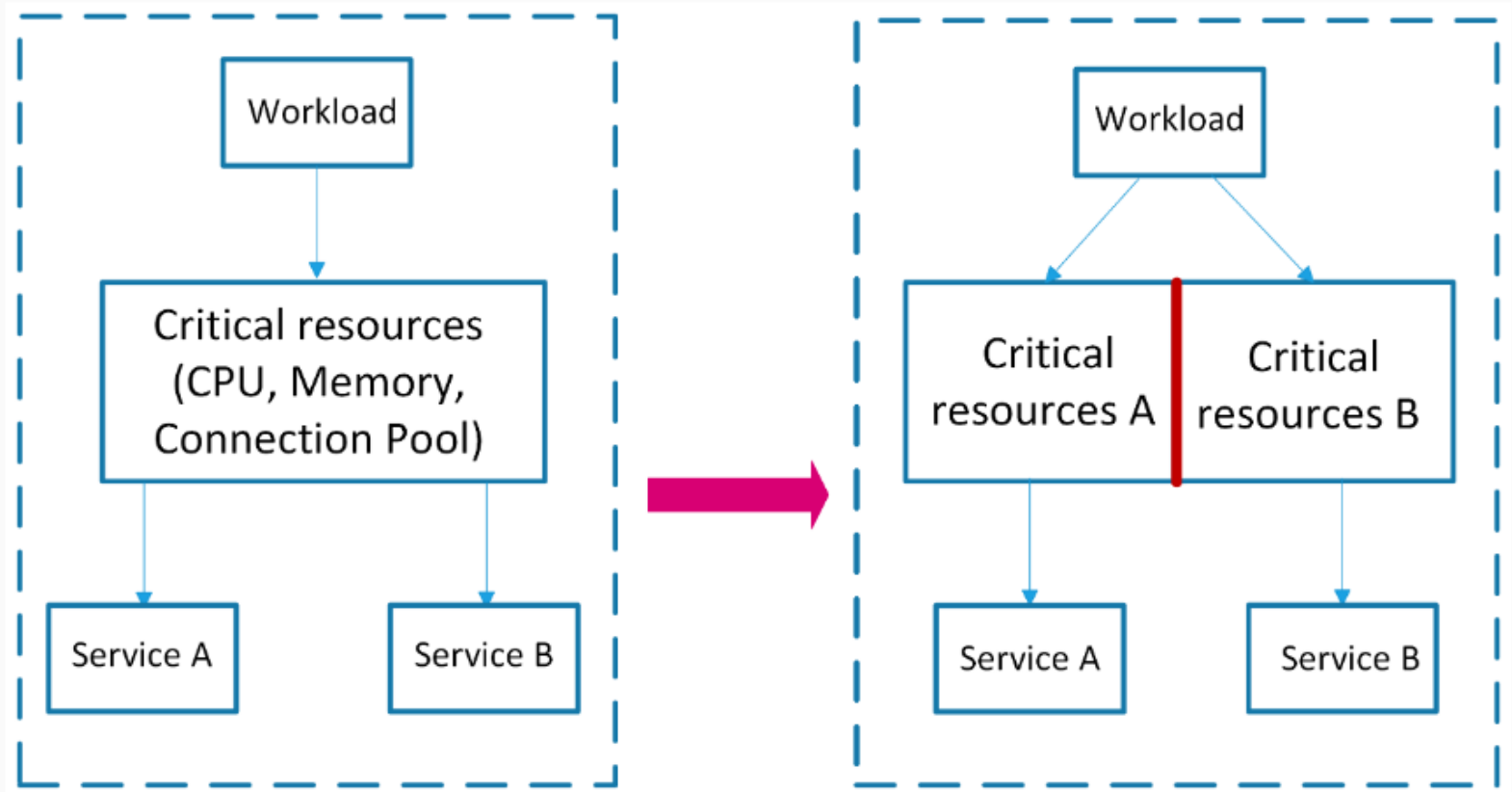
- При взаимодействии микросервисов не исключены ситуации, когда по какой-то причине один из сервисов перестает отвечать. Справиться с временными сбоями (медленное сетевое соединение, временная недоступность и так далее) помогают повторные вызовы. Однако при более серьезных сбоях, вызванных полным отказом сервиса, повторные вызовы будут лишь расходовать ресурсы.
- В таких случаях рекомендуется использовать шаблон Circuit Breaker. Микросервис будет запрашивать другой микросервис через Proxy-сервер. Он подсчитывает количество недавних сбоев и на основе него определяет, разрешать ли выполнение последующих вызовов или немедленно возвращать исключение.

Шаблон «Автоматический выключатель» (Circuit Breaker)



- Proxy-сервер может находиться в трех состояниях:
- **Closed.** Идет передача запросов между сервисами и подсчет количества сбоев. Если число сбоев за заданный интервал времени превышает пороговое значение, выключатель Proxy-сервера переводится в состояние Open.
- **Open.** Запросы от исходного сервиса немедленно возвращаются с ошибкой. По истечении заданного тайм-аута выключатель переводится в состояние Half-Open.
- **Half-Open.** Выключатель пропускает ограниченное количество запросов от исходного сервиса и подсчитывает число успешных запросов. Если необходимое количество достигнуто, выключатель переходит в состояние Closed, если нет — возвращается в статус Open.
- Использование шаблона повышает отказоустойчивость и предотвращает каскадные сбои, но требует тщательной настройки и мониторинга.

Шаблон «Переборка» (Bulkhead)

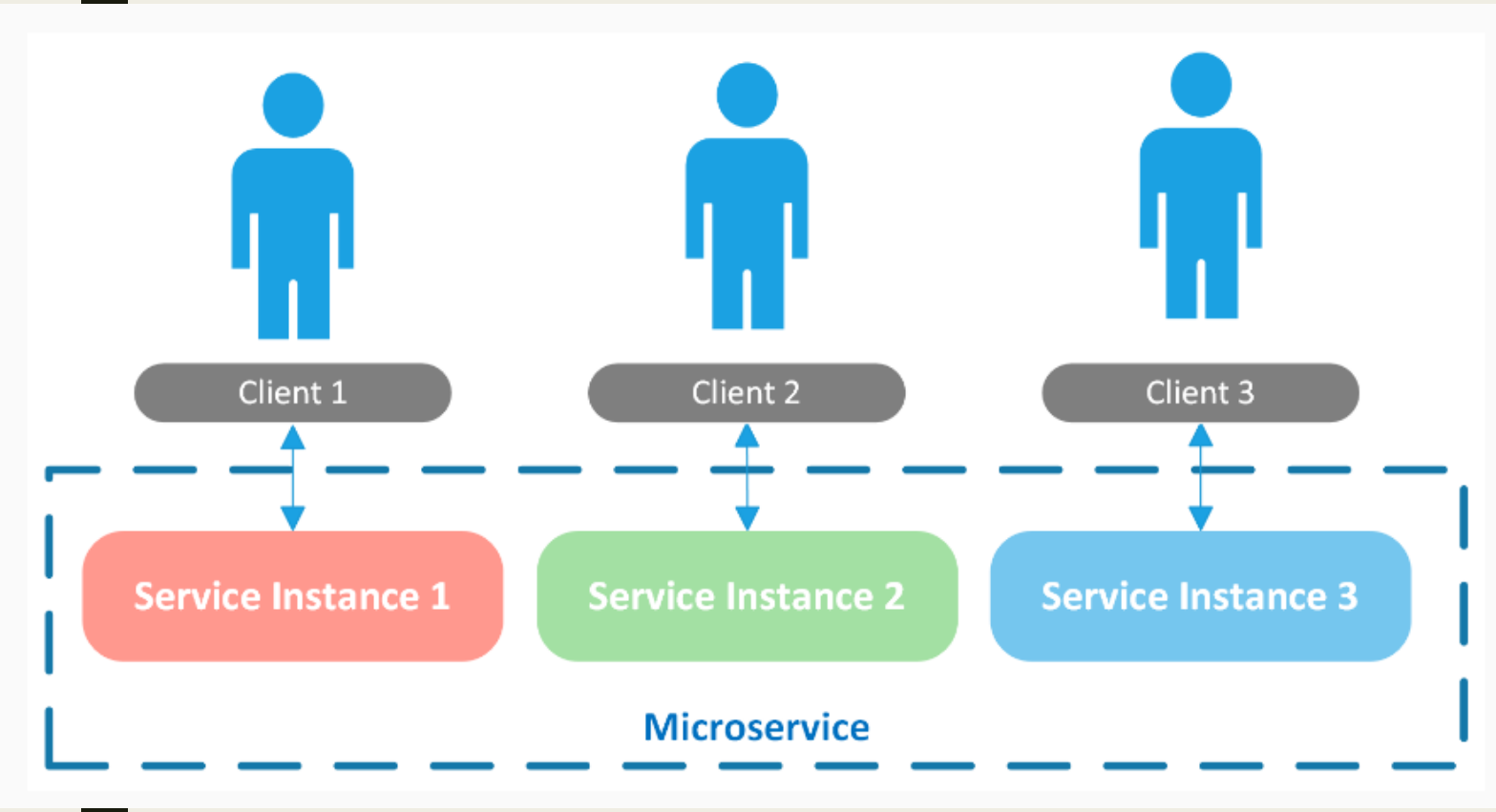


Вариант применения паттерна Bulkhead – разделение ресурсов, например пула соединений, между сервисами

Свое название паттерн получил благодаря переборкам, используемым в судостроении: они защищают корабль от полного затопления в случае повреждения отдельных его частей. Так же и в архитектуре приложения: переборки изолируют элементы приложения в пулы, чтобы в случае сбоя одного из них остальные продолжали функционировать.

Шаблон позволяет разделить ресурсы, чтобы гарантировать, что ресурсы, используемые для вызова одного сервиса, не влияют на ресурсы, используемые для вызова другого сервиса. Пример — использование отдельного пула соединений для каждого из нижестоящих сервисов.

Шаблон «Переборка» (Bulkhead)



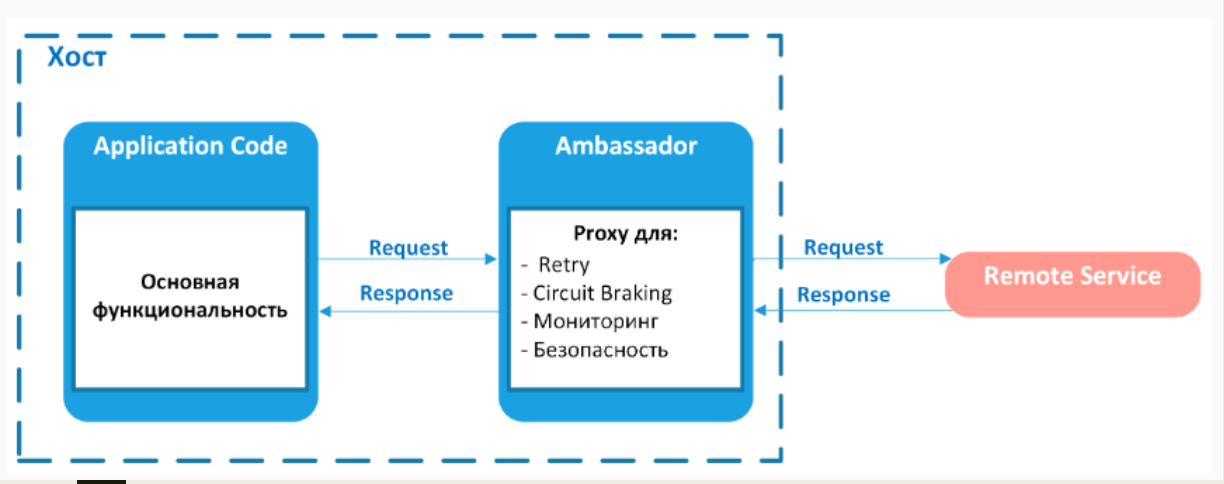
- Еще один вариант использования шаблона — назначение каждому клиенту сервиса отдельного экземпляра сервиса. В таком случае, если один из клиентов сделает слишком много запросов, перегрузив свой экземпляр, другие клиенты смогут продолжить работу.

Использование этого паттерна предотвращает каскадные сбои и изолирует критически важные ресурсы, но приводит к дополнительной сложности и менее эффективному использованию ресурсов.

Шаблон «Посредник» («Посол», Ambassador)

- Приложениям и сервисам часто требуются общие функции, относящиеся к мониторингу, ведению журналов, настройкам безопасности, сетевым службам и так далее. Однако в микросервисной архитектуре отдельные сервисы могут быть построены с помощью различных языков и технологий — следовательно, они могут иметь свои зависимости и требовать определенных языковых библиотек. Паттерн Ambassador предлагает помещать клиентские фреймворки и библиотеки для решения периферийных задач внутрь вспомогательного сервиса, выступающего в роли Proxy между клиентским приложением или основным сервисом и прочими частями системы.
- Так как добавление Proxy пусть и незначительно, но увеличивает сетевые задержки, шаблон Ambassador не рекомендуется использовать, когда время задержки критично. Также паттерн лучше не применять в случаях, когда можно обойтись стандартной клиентской библиотекой — например, если используется всего один язык или нет возможности выделить общие периферийные задачи.
- Развернуть Proxy можно как демон или службу. Если основной сервис является контейнерным, Proxy также разворачивается как отдельный контейнер на том же хосте, для этой цели используется другой паттерн — Sidecar.

Шаблон «Посредник» («Посол», Ambassador)



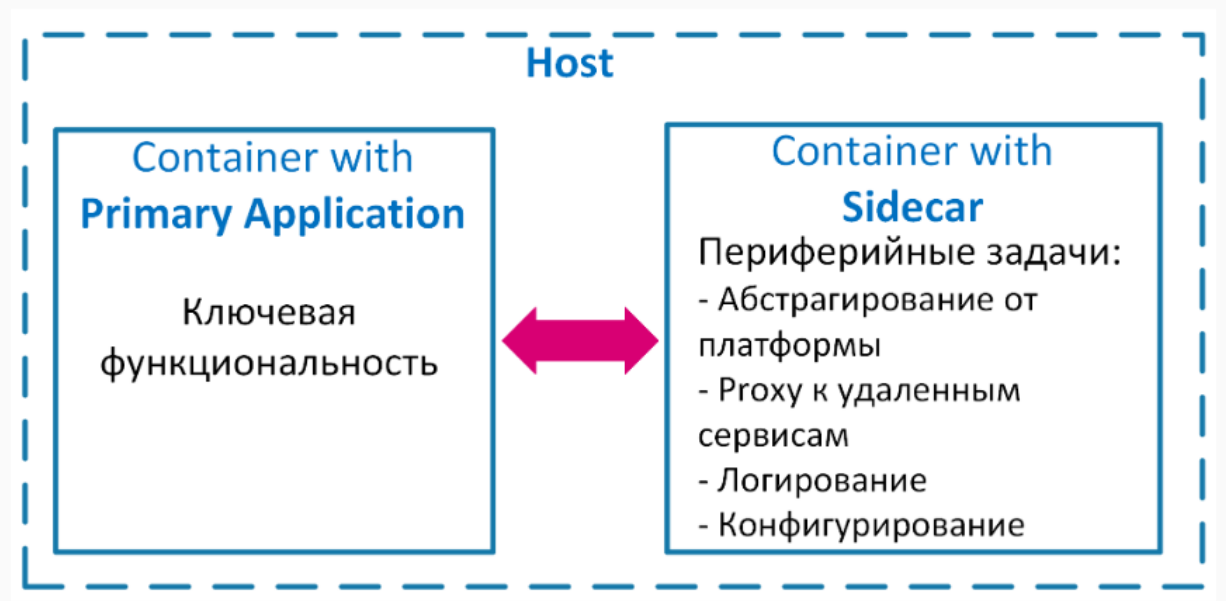
- Применение паттерна Ambassador позволяет:

Унифицировать обращение клиентских приложений к общим задачам независимо от используемого языка и фреймворка.

Решать периферийные задачи, не затрагивая основную функциональность, в том числе за счет передачи разработки отдельным специализированным командам. Это полезно, например, при необходимости централизованного управления сетевыми вызовами и функциями безопасности — во избежание дублирования сложного кода на каждом компоненте отдельно.

- Добавлять новую функциональность в Legacy-приложения, которые тяжело поддаются рефакторингу.

Шаблон «Коляска» («Прицеп», Sidecar)



- Паттерн Sidecar предлагает помещать периферийные задачи, связанные с мониторингом, безопасностью, отказоустойчивостью и так далее, в отдельный компонент и развертывать его внутри собственного процесса или контейнера. Так обеспечивается однородный интерфейс для сервисов основного приложения, которые могут быть написаны на разных языках.

Sidecar не обязательно является частью приложения, но связан с ним: для каждого экземпляра приложения рядом развертывается экземпляр Sidecar. Sidecar имеет тот же жизненный цикл, что и основное приложение.

Преимуществами паттерна являются независимость вспомогательного компонента от платформы основного приложения, возможность их доступа к одним и тем же ресурсам, минимизация задержек из-за их близкого расположения и возможность независимого обновления. К недостаткам можно отнести накладные расходы на создание дополнительного компонента. Шаблон не рекомендуется использовать для небольших приложений, а также в тех случаях, когда можно обойтись библиотеками и стандартными механизмами расширений.

Что есть у spring boot?



Все клево работает до spring boot 2.7. Потом есть некоторые моменты, которые работают не так, как описаны в книге.

Spring Cloud Service Discovery

Шаблон Spring Cloud Service Discovery помогает организовать обнаружение служб клиентами независимо от физического местоположения (IP-адреса и/или имени) ваших серверов. Клиенты вызывают бизнес-логику, используя логическое имя вместо физического адреса. Spring Cloud Service Discovery также предусматривает регистрацию экземпляров службы при их запуске и отмену регистрации по завершении работы. Реализовать Spring Cloud Service Discovery можно с помощью следующих служб:

- Consul (https://www.consul.io/);
- Zookeeper (https://spring.io/projects/spring-cloud-zookeeper);
- Eureka (https://github.com/Netflix/eureka) в качестве механизма обнаружения служб.

ПРИМЕЧАНИЕ. Проекты Consul и Zookeeper обладают весьма широкими возможностями, однако сообщество разработчиков на Java по-прежнему предпочитает использовать Eureka. В примерах в этой книге мы будем использовать Eureka, чтобы не распыляться и охватить как можно большую аудиторию. Тем, кого интересует Consul или Zookeeper, я советую заглянуть в приложения C и D. В приложении C вы найдете пример использования Consul для обнаружения служб, а в приложении D – пример использования Zookeeper.

Pom.xml

```
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud
    </groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server
    </artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-ribbon</artifactId>
        </exclusion>
    <exclusion>
        <groupId>com.netflix.ribbon</groupId>
        <artifactId>ribbon-eureka</artifactId>
    </exclusion>
</exclusions>
```

Подключить клиента, который юудет соединяться с Spring Config Server для получения настроек приложения

Подключить библиотеки Eureka

Исключить библиотеки Netflix Ribbon

Pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
```



```
  <artifactId>spring-cloud-starter-loadbalancer
  </artifactId>
</dependency>
```

Подключить библиотеки Spring Cloud Load Balancer

Eureka

Листинг 6.3. Настройки Eureka в Spring Config Server

```
server:
    port: 8070
eureka:
    instance:
        hostname: localhost
    client:
        registerWithEureka: false
        fetchRegistry: false
        serviceUrl:
            defaultZone:
                http://${eureka.instance.hostname}:${server.port}/eureka/
    server:
        waitTimeInMsWhenSyncEmpty: 5
```

Порт, прослушиваемый сервером Eureka Server

Имя хоста, где находится экземпляр Eureka

Запретить серверу Config Server регистрироваться в службе Eureka ...

.... и не кешировать информацию из реестра

URL службы

Начальное время ожидания, прежде чем сервер начнет принимать запросы

Ключевыми в листинге 6.3 являются следующие свойства:

- `server.port` – задает порт по умолчанию;
- `eureka.instance.hostname` – имя хоста, где размещается экземпляр службы Eureka;
- `eureka.client.registerWithEureka` – запрещает серверу Config Server регистрироваться в Eureka при запуске приложения Spring Boot Eureka;
- `eureka.client.fetchRegistry` – значение `false` запрещает службе Eureka кешировать информацию из реестра. Это значение следует изменить для служб Spring Boot, которые должны регистрироваться в Eureka;
- `eureka.client.serviceUrl.defaultZone` – URL службы для любого клиента. Это комбинация атрибутов `eureka.instance.hostname` и `server.port`;
- `eureka.server.waitTimeInMsWhenSyncEmpty` – время ожидания перед тем, как сервер начнет принимать запросы.

Main-класс

Листинг 6.4. Аннотации поддержки Eureka Server для класса инициализации

```
package com.optimagrowth.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Разрешает приложению Eureka Server работать в роли реестра служб Spring

Для клиента

Листинг 6.5. Добавление зависимости Spring Eureka в файл pom.xml службы организаций

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>
        spring-cloud-starter-netflix-eureka-client
    </artifactId>
</dependency>
```

Подключить библиотеки Eureka, чтобы служба могла зарегистрировать себя в Eureka

Листинг 6.6. Добавление spring.application.name в настройки службы организаций

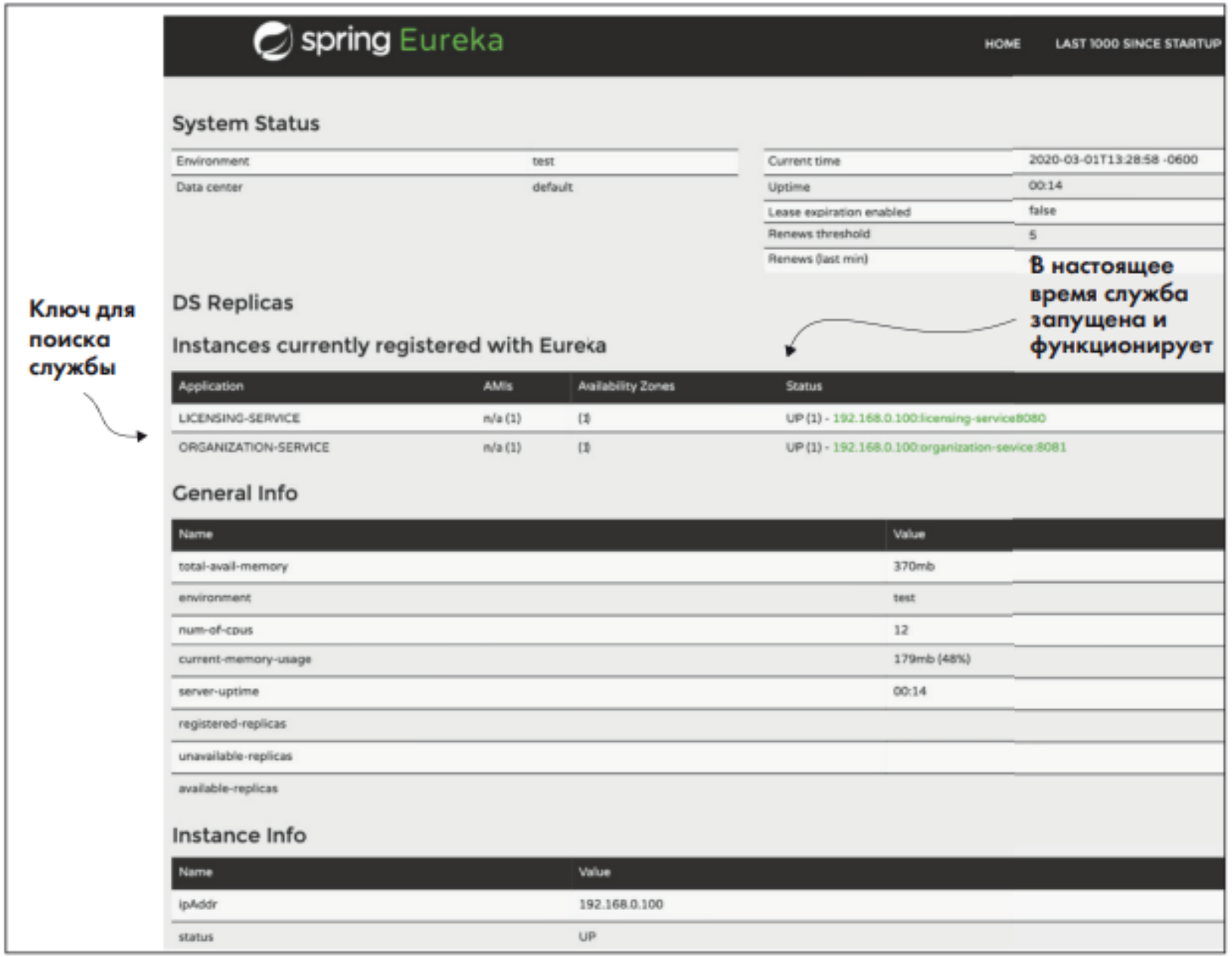
```
spring:
    application:
        name: organization-service
        profiles:
            active: dev
```

Логическое имя службы, которое будет зарегистрирован в Eureka

Для клиента

```
eureka:
    instance:
        preferIpAddress: true
    client:
        registerWithEureka: true
        fetchRegistry: true
        serviceUrl: defaultZone: http://localhost:8070/eureka/
```

Итог


spring Eureka
HOME
LAST 1000 SINCE STARTUP
System Status
Environment test
Data center default
Current time 2020-03-01T13:28:58 -0600
Uptime 00:14
Lease expiration enabled false
Renews threshold 5
Renews (last min)
В настоящее время служба запущена и функционирует
Ключ для поиска службы
DS Replicas
Instances currently registered with Eureka
Application AMIs Availability Zones Status
LICENSING-SERVICE n/a (1) (1) UP (1) - 192.168.0.100:licensing-service:8080
ORGANIZATION-SERVICE n/a (1) (1) UP (1) - 192.168.0.100:organization-sevice:8081
General Info
Name Value
total-avail-memory 370mb
environment test
num-of-cpus 12
current-memory-usage 179mb (48%)
server-uptime 00:14
registered-replicas
unavailable-replicas
available-replicas
Instance Info
Name Value
ipAddr 192.168.0.100
status UP

Код клиента

Листинг 6.11. Подготовка класса инициализации для использования Eureka Discovery Client

```
package com.optimagrowth.license;

@SpringBootApplication
@RefreshScope
@EnableDiscoveryClient
public class LicenseServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(LicenseServiceApplication.class, args);
    }
}
```

Активирует Eureka Discovery Client

Отправка запроса

Листинг 6.12. Использование Discovery Client для поиска информации

```
@Component
public class OrganizationDiscoveryClient {

    @Autowired
    private DiscoveryClient discoveryClient;

    public Organization getOrganization(String organizationId) {
        RestTemplate restTemplate = new RestTemplate();
        List<ServiceInstance> instances =
            discoveryClient.getInstances("organization-service");

        if (instances.size()==0) return null;
        String serviceUri = String.format
                ("%s/v1/organization/%s",
                instances.get(0).getUri().toString(),
                organizationId);
        ResponseEntity<Organization> restExchange =
            restTemplate.exchange(
                serviceUri, HttpMethod.GET,
                null, Organization.class, organizationId);
        return restExchange.getBody();
    }
}
```

Внедряет Discovery Client в класс

Получает список всех экземпляров службы организаций

Получает конечную точку службы

Использует стандартный класс RestTemplate для вызова службы

Feign Client

Листинг 6.15. Использование клиента Spring Cloud/Netflix Feign в службе лицензий

```
@SpringBootApplication
@EnableFeignClients
public class LicenseServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(LicenseServiceApplication.class, args);
    }
}
```

Эта аннотация необходима для использования клиента Feign

FeignClient

Листинг 6.16. Определение интерфейса Feign для вызова службы организаций

```
// Объявление пакета и инструкции импорта опущены для краткости
@FeignClient("organization-service")
public interface OrganizationFeignClient {
    @RequestMapping(
        method= RequestMethod.GET,
        value="/v1/organization/{organizationId}",
        consumes="application/json")
    Organization getOrganization
        (@PathVariable("organizationId")
            String organizationId);
}
```

Определение службы в Feign

Определение пути к конечной точке и операции с ней

Определение параметров для передачи конечной точке

Пример: обмен без эврики с restTemplate